

Module 8: Trees

Part 1: Basic Decision Trees

Wenting Xu

Iowa State University

Copyright and Attribution

Some lecture slides and instructional materials in this course are adapted from the following sources:

- *An Introduction to Statistical Learning: With Applications in R (Second Edition)*

Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani
Springer, 2021

- Online course materials developed by Trevor Hastie, Robert Tibshirani, and collaborators.

Tree-based methods

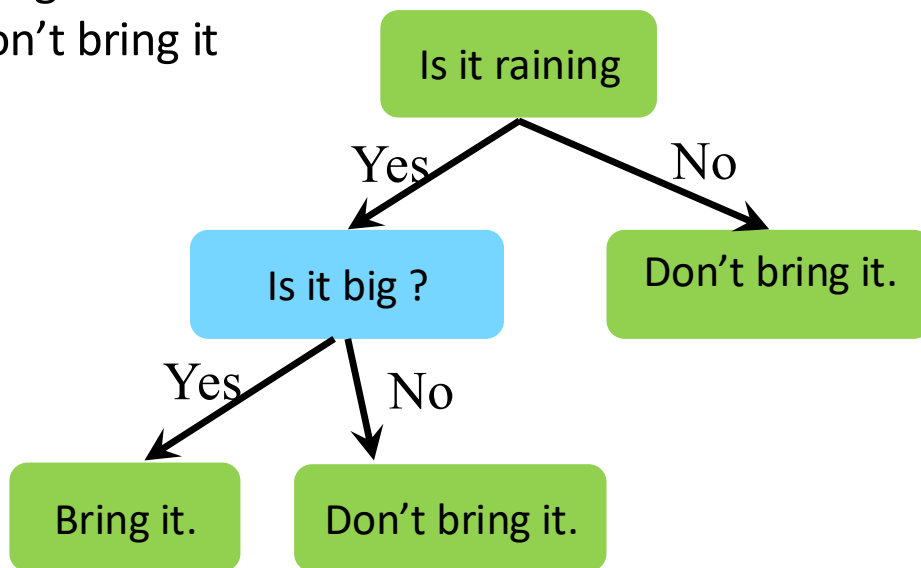
- Tree-based methods are supervised learning methods that can be used for regression and classification.
- These methods involve stratifying or segmenting the predictor space into a number of simple regions.
- Tree-based methods are simple and useful for interpretation.
- However, they typically are not competitive with the best supervised learning approaches.
- Instead, we use methods that involve combining multiple trees: bagging, random forests, and boosting.

How does decision tree work ?

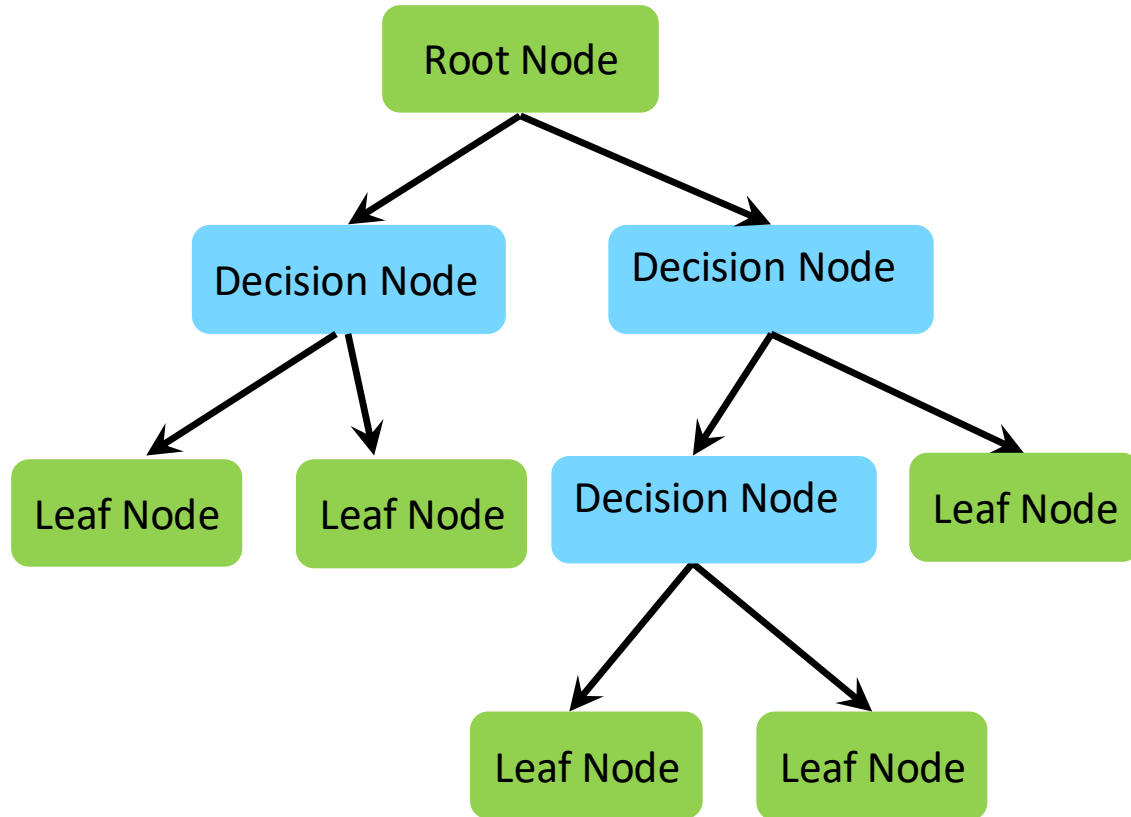
You're about to leave the house and need to decide:

Option 1: Bring an umbrella

Option 2: Don't bring it

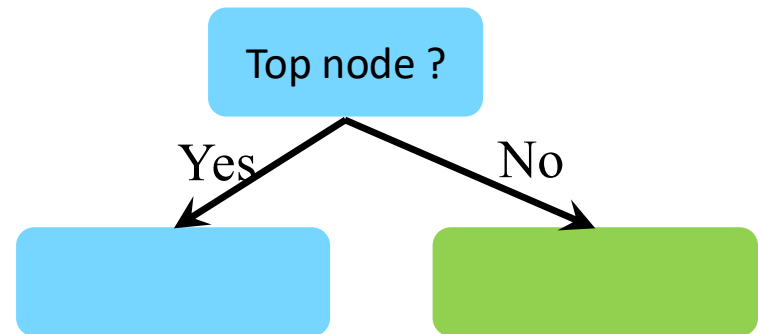


How does decision tree work ?



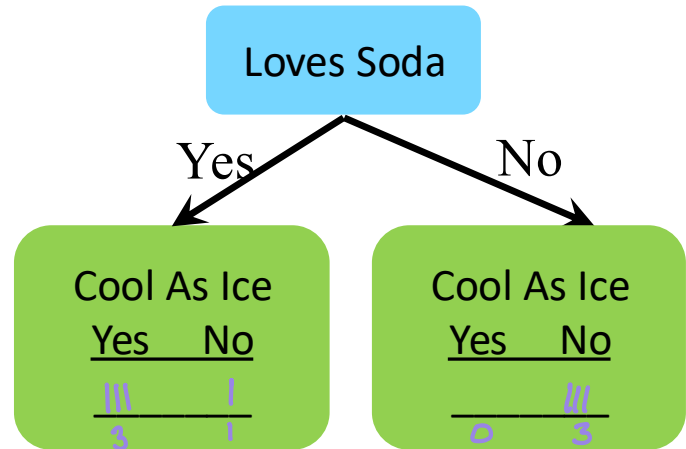
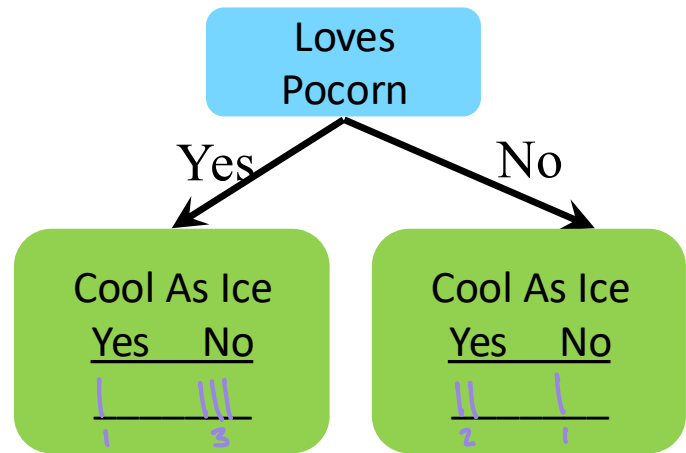
Decision Tree Example: Predicting “Loves Cool Ice”

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



Decision Tree Example: Predicting “Loves Cool Ice”

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No
No	Yes	24	Yes



Decision Tree Example: Predicting “Loves Cool Ice”

Gini Impurity

$$= 1 - (\text{the probability of “Yes”})^2 - (\text{the probability of “No”})^2$$

Gini impurity for leaf **Loves popcorn Yes**

$$= \frac{1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2}{}$$
$$= 0.375$$

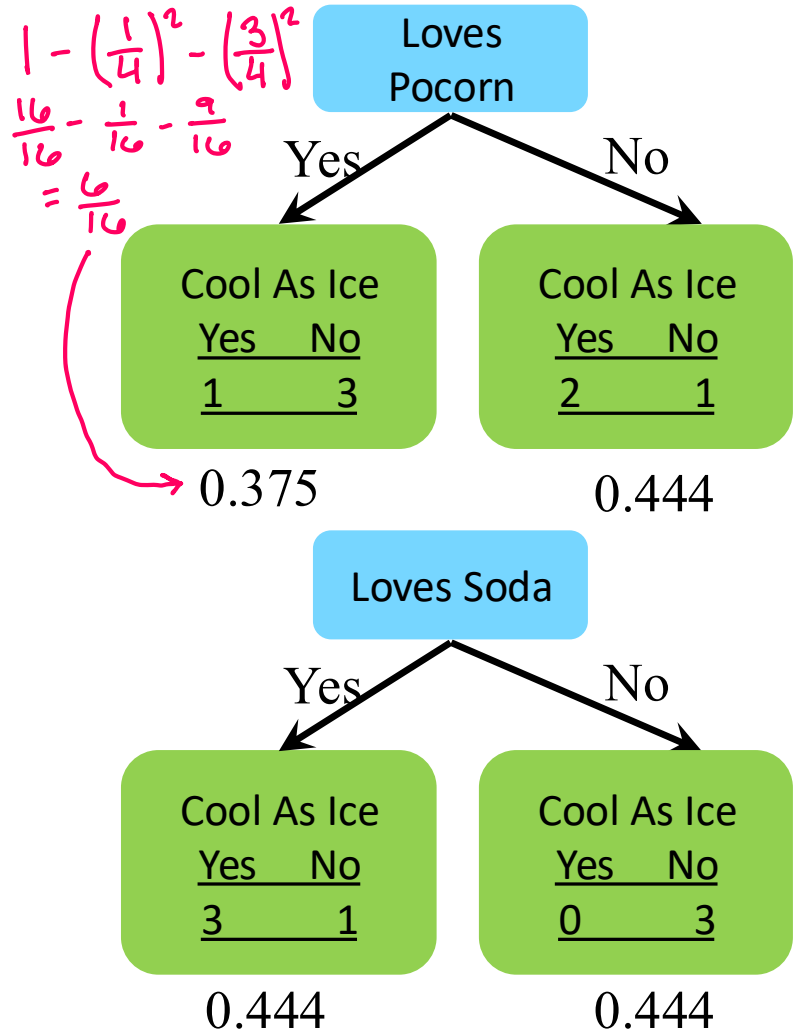
Gini impurity for leaf **Loves popcorn No**

$$= \frac{1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2}{} = 0.444$$

Gini impurity for leaf **Loves popcorn Yes**

= weighted average of Gini Impurities for the Leaves

$$= \frac{4}{7} (0.375) + \frac{3}{7} (0.444) = 0.405$$

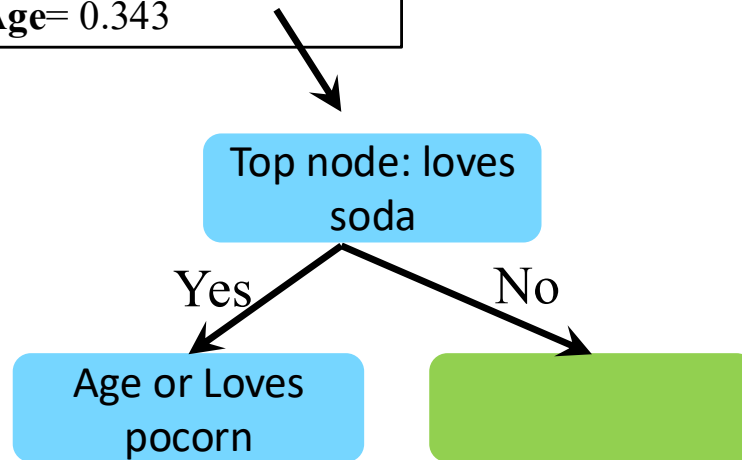


Decision Tree Example: Predicting “Loves Cool Ice”

Gini impurity for **Loves popcorn** = 0.405

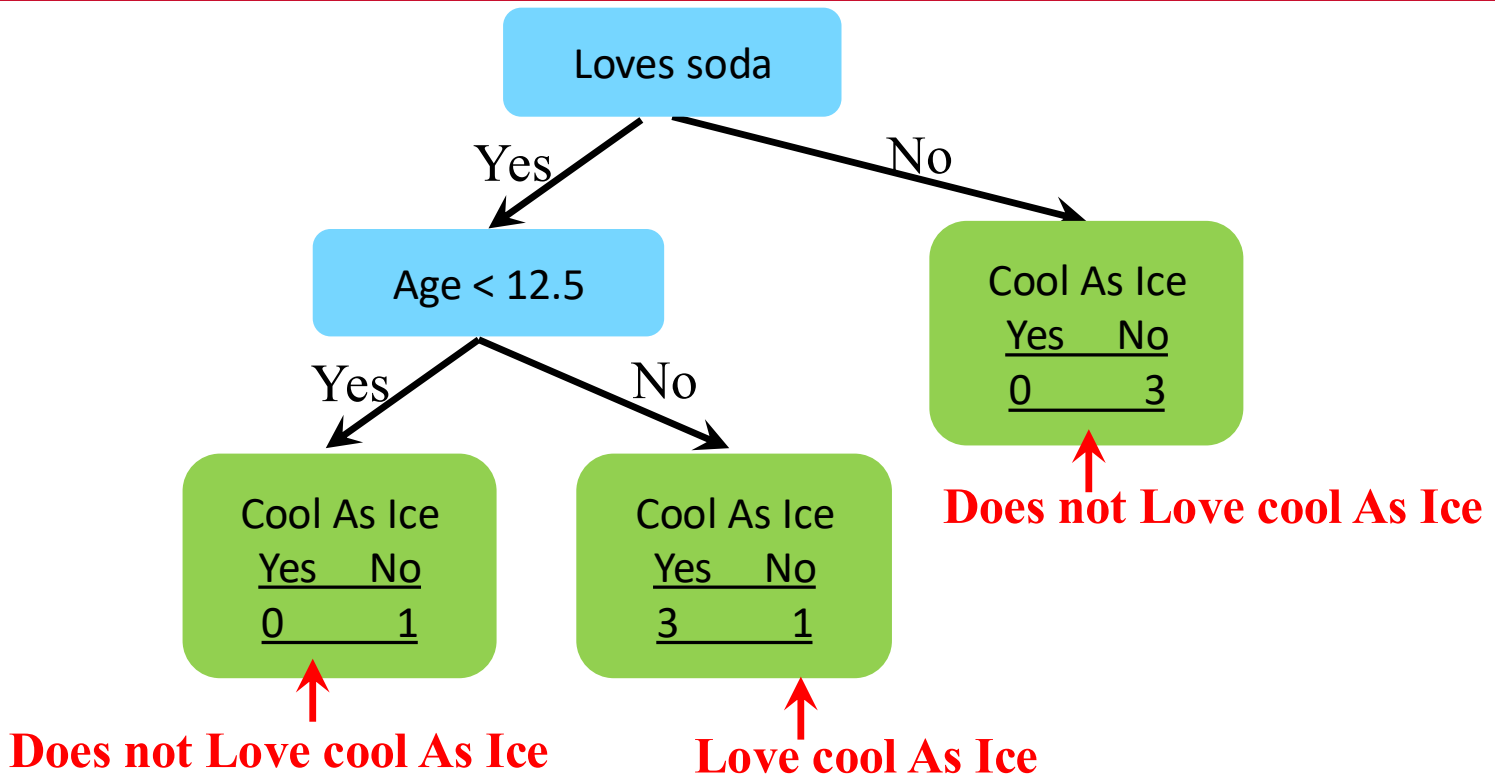
Gini impurity for Loves soda = 0.214

Gini impurity for **Age** = 0.343



- Focus on the current node (e.g., **Love Soda = Yes**)
- Consider remaining features (e.g., **Age, Loves Popcorn**)
- For each feature: Compute the **Gini impurity** of the resulting groups
- **Select the feature with the lowest Gini impurity** (most pure split)
Age is selected (Age = 0 , Popcorn = 0.25)

Decision Tree Example: Predicting “Loves Cool Ice”

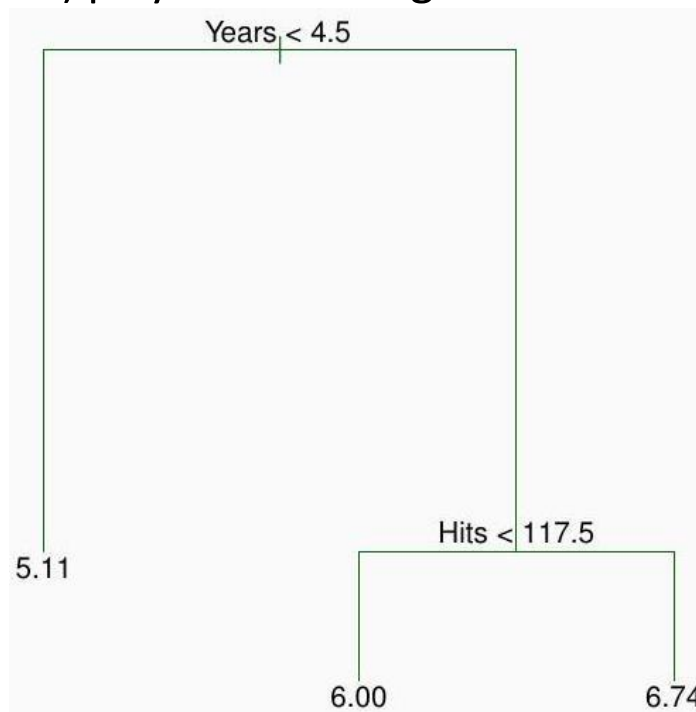


Predict:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	15	????

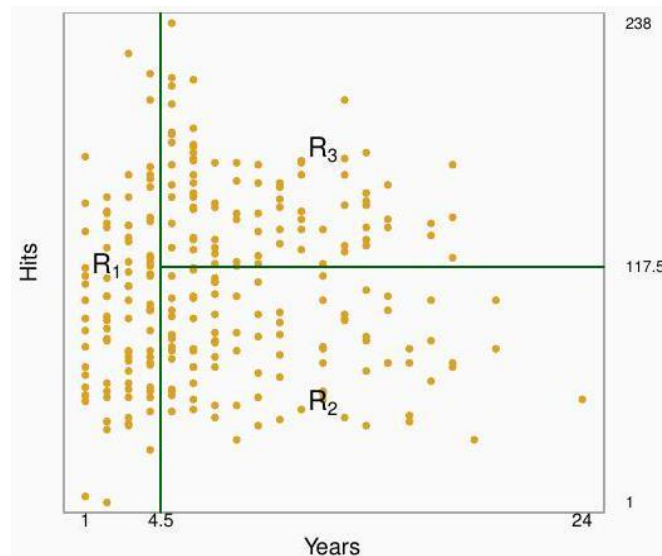
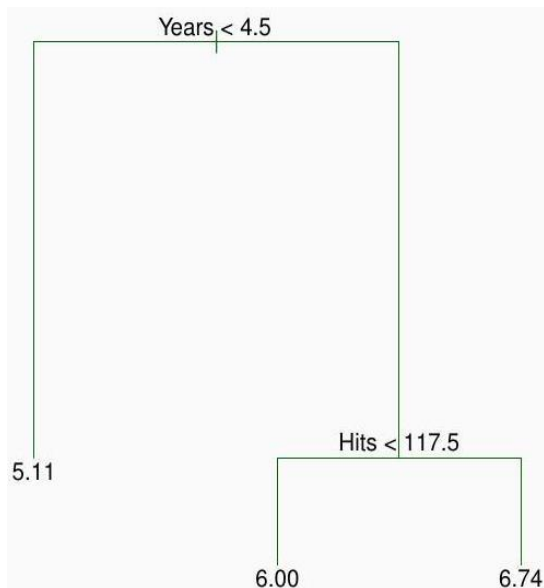
Regression Trees

- Example: Hitters dataset. We want to predict a baseball player's **Salary** based on Years and Hits. Salary here is log-transformed so it is more bell-shaped.
- The tree segments (or segments) players into 3 regions of the predictor space:



Regression Trees

- Regions R_1, R_2, R_3 are known as terminal nodes or leaves of the tree.
- Points along the tree where the predictor space is split are referred to as internal nodes.
- Segments of the trees that connect the nodes are known as branches.



Prediction via Segmentation of the Predictor Space

Regression tree building process:

1. We divide the predictor space (the set of possible values for $X_1, X_2, X_3, \dots, X_p$, into J distinct and non-overlapping regions: $R_1, R_2, \dots, R_J$.
2. For every observation that falls into the region R_j , we make the same prediction, which is simply the **mean of the response values** for the training observations in R_j .

How do we construct the regions $R_1, \dots, R_J$?

- In theory, regions could have any shape.
- However, we choose to divide the predictor space into boxes for simplicity and easy of interpretation.
- Then the goal is to find boxes $R_1, \dots, R_J$ that minimize the RSS, given by:

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

where $\hat{y}_{R_j}$ is the mean response for the training observations within the j th box.

How do we construct the regions $R_1, \dots, R_J$?

- Computationally infeasible to consider every partition of the feature space into J boxes.

Recursive binary splitting

- Top-down recursive greedy approach.
- Top-down recursive: start with all observations and split into two branches at each level of the tree.
- Greedy: best split is made at each step without looking ahead.

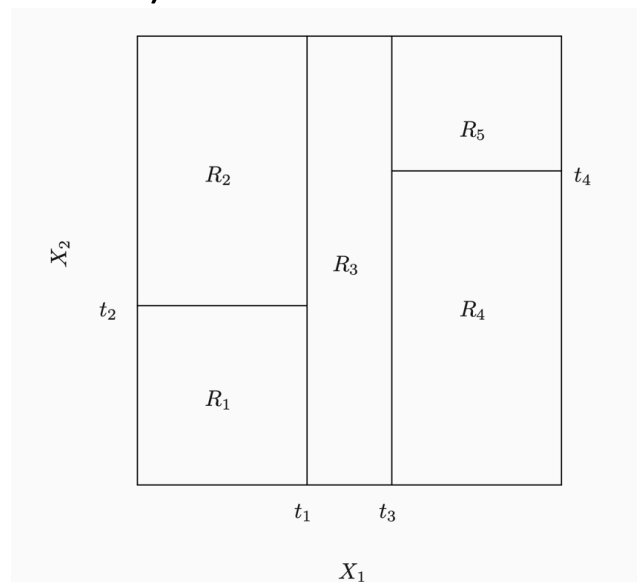
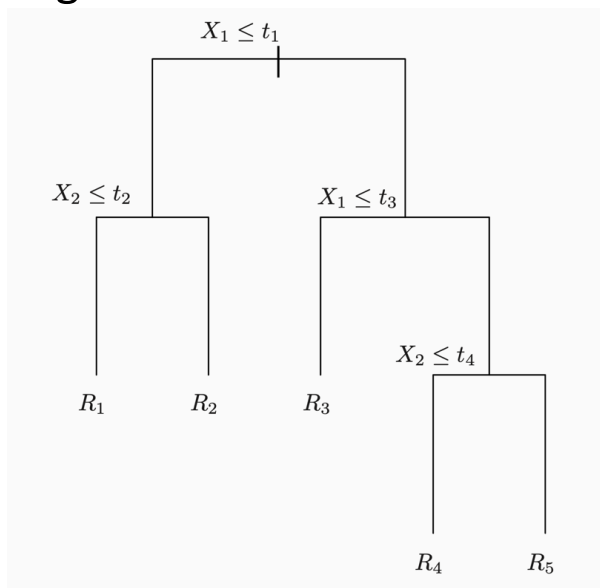
Recursive binary splitting

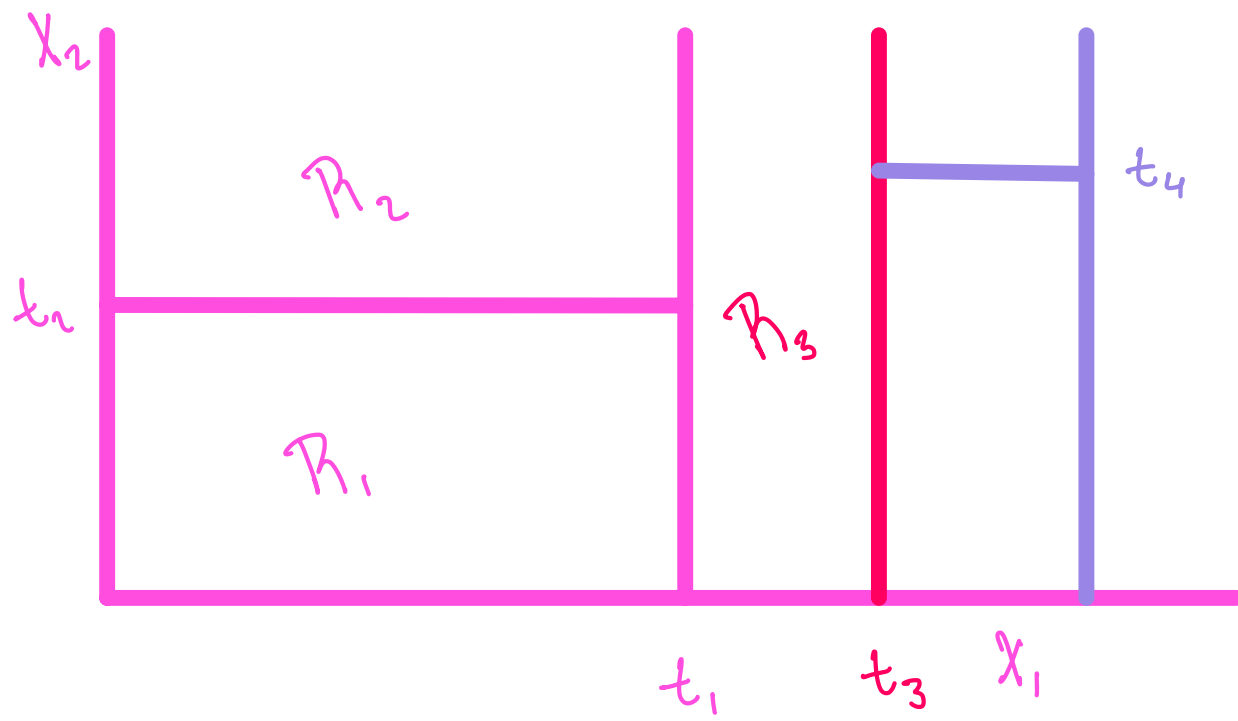
Choose a predictor X_j and a cutpoint s that minimizes the training RSS for the resulting tree:

$$R_1(j, s) = \{X \mid X_j < s\}, R_2(j, s) = \{X \mid X_j \geq s\}$$
$$RSS = \sum_{x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

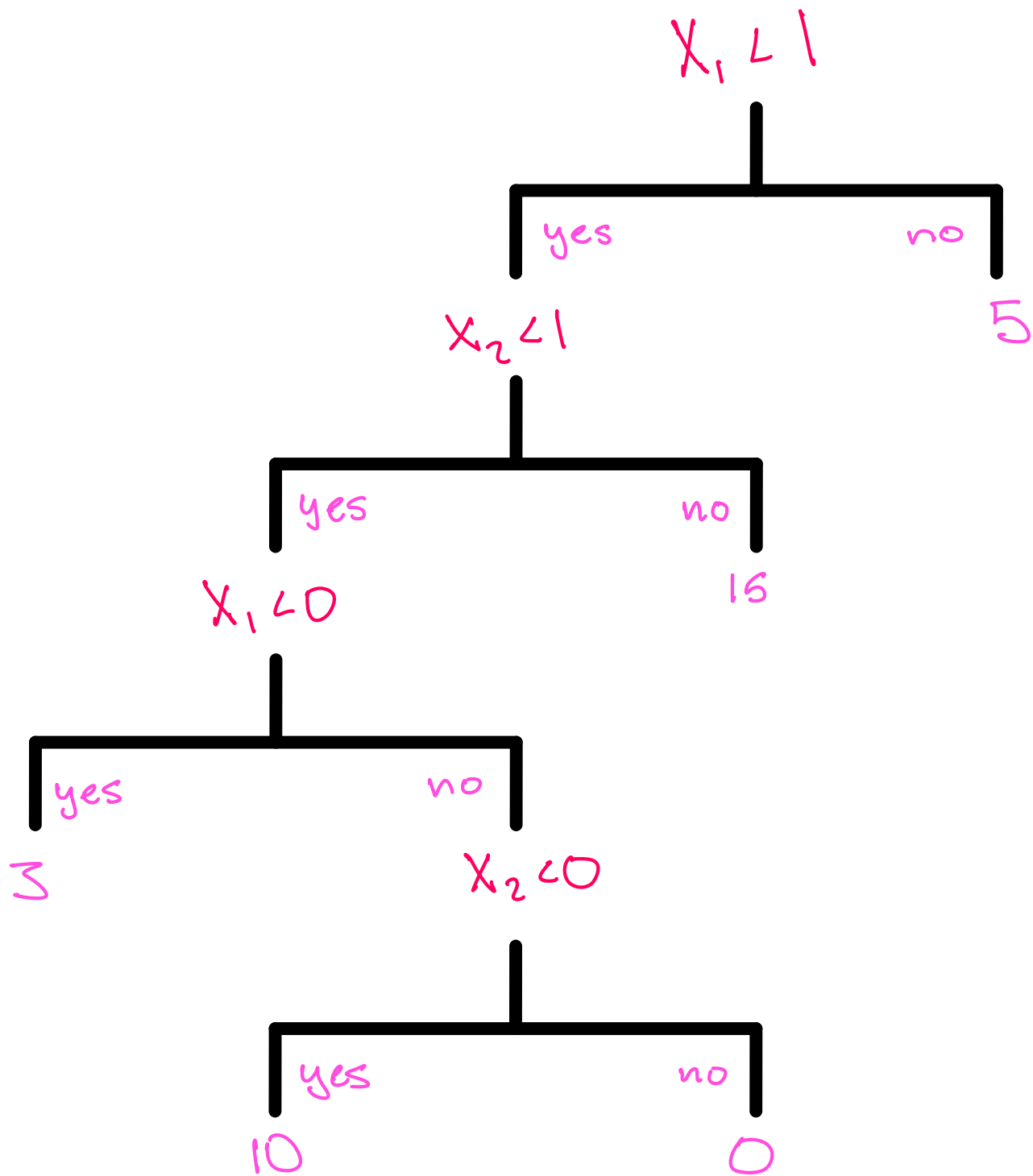
Recursive binary splitting

- Repeat the process: look for the best predictor and best cutpoint in order to split the data further so as to minimize the RSS within each resulting region.
- Now instead of splitting entire predictor space, we split one of the previously identified regions.
- This process continues until a stopping criteria is reached (i.e. no regions contains more than 5 observations).

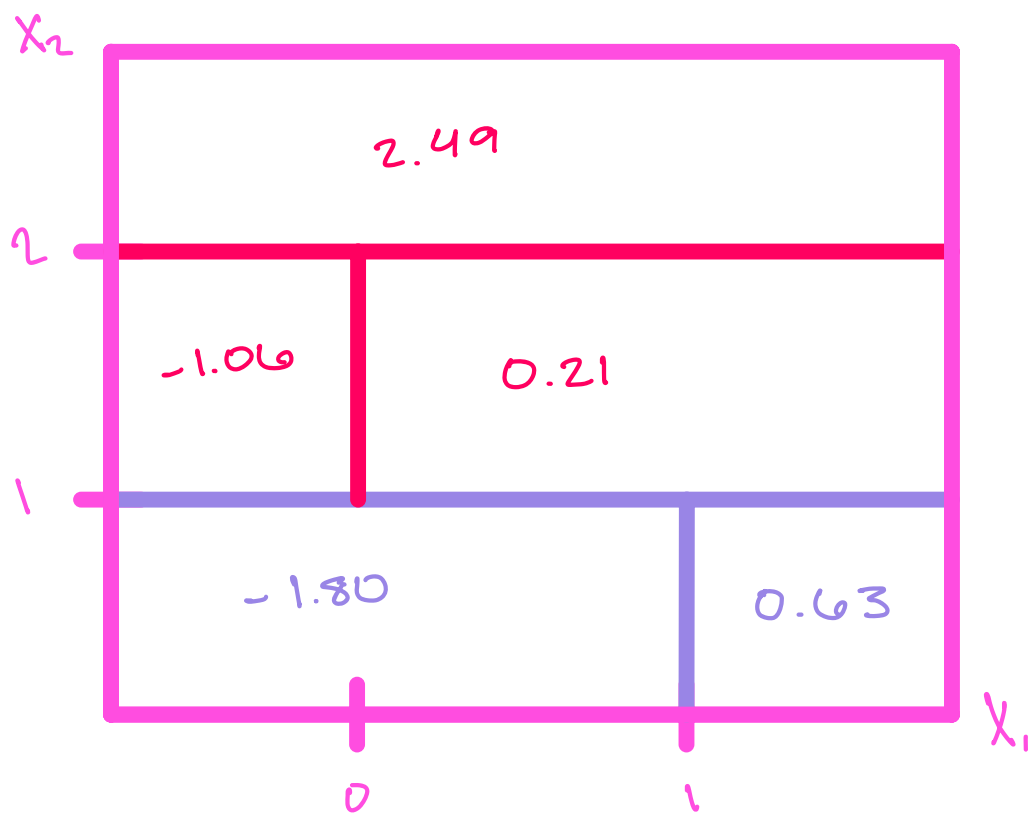
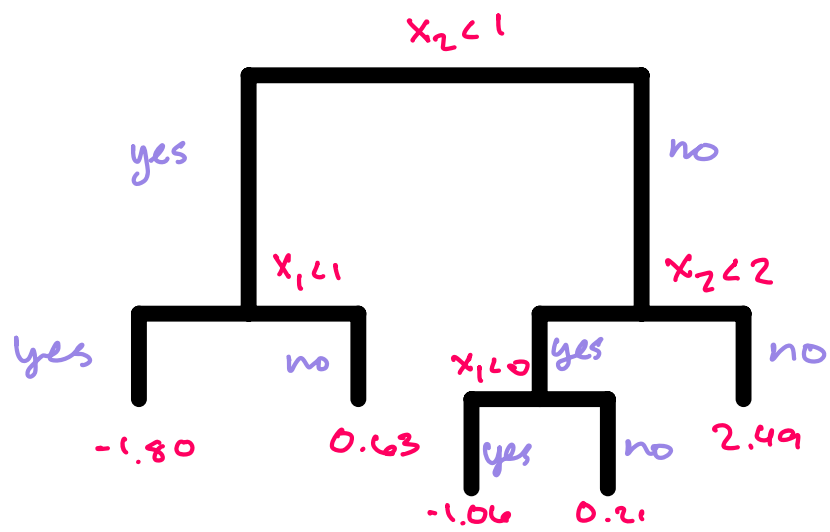




Homework 12 Q1a thoughts



Homework 12 answer (part 1b)



Tree Pruning

- The process we described is a good starting point
 - However, it can **overfit** the training data
 - This may lead to poor performance on test data
-
- The tree can become too complex
 - Too many splits $\rightarrow$ too many regions $R_1, R_2, \dots, R_J$
 - The model captures noise instead of true patterns
-
- Our strategy: **grow a very large tree T_0 and then prune it back in order to obtain a smaller tree (subtree).**

Cost complexity pruning

- Idea: we apply a **penalty** to the tree.
- For every value of α , there is a subtree $T \subset T_0$ such that:

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

is as small as possible.

- $|T|$ denotes the number of leaves in the tree.
- α is a tuning parameter: controls the trade-off between the subtree's complexity and its fit to the training data.
- When $\alpha = 0$, we just have the original tree T_0 .
- When α increases, what happens to our subtree?
- Reminiscent of lasso.

Algorithm for building a regression tree

1. Use recursive binary splitting to grow a large tree on the training data, stopping according to a criteria.
2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α .
3. Use K -fold cross-validation to choose α . For each $k = 1, \dots, K$:
 - a) Repeat Steps 1 and 2 on all but the k th fold of the training data.
 - b) Evaluate the mean squared prediction error on the data in the **left-out k th fold**. Average the results for each value of α and pick α to minimize the k -fold CV error.
4. Return the subtree from Step 2 that corresponds to the chosen value of α .

See R script: `decisiontreesbasics.R`